

Georgia State University

CSC 4780/6780 & DSCI 4780 – Fundamentals of Data Science

Fall 2024

Project Progress Report

Hospital Readmission Prediction for Diabetes Patients

RecallMe

Pranjal Patil

Natnael Alemayehu

Kushal Sarkar

Kevin Gallardo

Table of Contents

1 Business Understanding.....	3
1.1 Business Problem.....	3
1.2 Dataset.....	3
1.3 Proposed Analytics Solution.....	4
2 Data Exploration and Preprocessing.....	5
2.1 Data Quality Report	5
2.2 Missing Values and Outliers	9
2.2.1 Data Cleaning and Dropping Features	9
2.2.2 Outlier Detection.....	10
2.2.3 Handling Missing Data	10
2.2.4 Duplicate Removal:.....	11
2.2.5 Handling Outliers.....	11
2.3 Normalization.....	12
2.3.1 Normalization Method	12
2.3.2 Normalized Features	12
2.3.3 Impact of Normalization	13
2.4 Transformations	13

1 Business Understanding

There have been many advancements in the realm of diabetes treatment, but most patients are not taking advantage of those treatments. This could be exacerbated by the hospital's management of patients with diabetes which could be improved to allow patients to reach a stable condition before being discharged and prevent unnecessary suffering of patients and even deaths. By being able to better predict the early readmission of patients within 30 days of discharge we can improve the management of hospitals by allowing more time for the patients to stabilize and avoid the costs of readmissions, and any complications that could happen from being discharged too early. We choose the dataset of 130-US hospitals for the years 1999 to 2008 available at UC Irvine machine learning repository. The dataset has a total of 47 descriptive features, with some of them being admission type, number of lab procedures, whether certain drugs were prescribed or not, and the target feature of readmitted. The dataset has a total of 101,766 instances occurring within a span of 10 years. One of the proposed analytics solutions is a readmission prediction model. Our solution is to use a supervised learning model to classify if a patient will be readmitted within 30 days. An additional goal would be to determine the most impactful factors that lead to an early readmission within 30 days of discharge to provide possible new insights into the proper discharge of patients with diabetes.

1.1 Business Problem

Managing diabetes effectively remains a significant challenge, with many patients not leveraging advancements in treatment. This situation may be exacerbated by hospitals discharging patients prematurely, leading to avoidable suffering, complications, or even deaths. One way to address this issue is to predict early readmissions within 30 days of discharge, which would allow hospitals to manage patients better. By ensuring that patients have stabilized before being discharged, healthcare facilities can prevent unnecessary readmissions, reduce associated costs, and improve patient outcomes.

1.2 Dataset

The dataset consists of patient records from 130 U.S. hospitals and integrated delivery networks, spanning a 10-year period (1999–2008). It includes over 50 features representing various aspects of patient and hospital outcomes. The dataset specifically focuses on inpatient encounters for patients diagnosed with diabetes, including those who had laboratory tests performed, and medications administered during their stay.

Key Characteristics of the Dataset:

- I. Inpatient Encounters: The data is restricted to inpatient encounters where the patient was admitted to the hospital, fulfilling the criteria for hospitalization, including a length of stay between 1 and 14 days.
- II. Diabetic Encounters: Each record represents an encounter where a diabetes diagnosis (of any

type) was entered into the system.

- III. **Medical Interventions:** The dataset includes details of medical procedures such as laboratory tests and medications administered during the hospitalization. These features are crucial in understanding the clinical care provided to the patients.
- IV. **Demographic Features:** It contains sensitive information like age, gender, and race, which are important factors in diabetes care but must be handled carefully due to privacy concerns.
- V. **Hospital and Treatment Details:** The dataset also includes information on the admission type, medical specialty of the admitting physician, number of procedures and lab tests, medications, and patient health outcomes like HbA1c test results.

Features of the Dataset:

- I. **Patient Demographics:** Age, gender, race, and patient number.
- II. **Admission Information:** Type of admission, time spent in the hospital, and admitting physician.
- III. **Medical Data:** Number of lab tests performed, results of key medical tests, number of medications, and diabetic medications used.
- IV. **Historical Data:** Information about prior hospital visits (outpatient, emergency, inpatient) within the past year.

This dataset is crucial for analyzing patient outcomes and predicting early readmissions for diabetic patients. The data spans a substantial time frame, which allows for historical analysis of changes in hospital practices and patient outcomes over the years.

P Sensitive Data:

The dataset includes sensitive personal information such as age, gender, and race, which are protected under healthcare data privacy regulations such as HIPAA in the U.S.

1.3 Proposed Analytics Solution

The proposed solution aims to develop a supervised machine learning model that predicts whether a diabetic patient will be readmitted to the hospital within 30 days of discharge. This predictive model will enable hospitals to optimize patient management, reduce readmission rates, and improve hospital resource allocation.

Key Objective:

Predict Early Readmissions: The primary objective is to predict whether a diabetic patient will be readmitted within 30 days. This will help identify patients who are at higher risk of complications and early discharge.

Impact on Hospital Management:

1. **Improved Management:** By predicting readmissions, hospitals can take timely action to ensure that patients have fully recovered and stabilized before being sent home, thus reducing unnecessary readmissions and associated costs.
2. **Enhance Patient Care:** Ensuring patients are stable before discharge will reduce the risk of readmissions and adverse outcomes, improving overall patient care.
3. **Optimize Resource Allocation:** Predictive models can help hospitals manage their resources more effectively, ensuring that the right number of staff and medical equipment are available for high-risk patients.

2 Data Exploration and Preprocessing

2.1 Data Quality Report

When exploring our attributes we saw that we had 13 continues attributes and 37 categorical attributes. Three of the continues attributes (admission_type_id, discharge_disposition_id, admission_source_id) were nominal attributes so we changed them to objects in the data frame. So, after that we have 10 continues attributes and 40 categorical attributes. The continues attributes include two id types (encounter_id and patient_nbr) and nine types of metrics (time in hospital, number of: lab procedures, procedures (not including lab procedures), medications, outpatient (visit), emergency, inpatient, and diagnosis(count). For the categorical features there is information about the patient like race, gender, age, and weight. There is also information about the payer code, medical specialty, admission type id, discharge_disposition_id, admission source id, and 3 types of diagnosis. Then there are 23 types of medicine that could be given to the patient. Then we have features about the patient's care like the glucose serum test result, A1c test result, if there was a change in medication, and if diabetes medication was prescribed or not. Lastly, we have our target feature which is the 'readmitted' feature. The readmitted feature which can take three types of values: <30 (readmitted within 30 days), >30(readmitted after 30 days), and No. When we first upload the data and view the data frame, it becomes apparent that some fields are filled in with '?'. So, the first thing we did is replace the question marks with a 'nan' value. We also noticed that the gender had a 'Unknown/Invalid' value, so we replaced that with a 'nan' value as well.

Table 1. Data Quality Report for Continuous Features

Description of Dataset for Numeric Features:

	count	mean	std	min	25%	50%	75%	max
encounter_id	101766.0	1.652016e+08	1.026403e+08	12522.0	84961194.0	152388987.0	2.302709e+08	443867222.0
patient_nbr	101766.0	5.433040e+07	3.869636e+07	135.0	23413221.0	45505143.0	8.754595e+07	189502619.0
time_in_hospital	101766.0	4.395987e+00	2.985108e+00	1.0	2.0	4.0	6.000000e+00	14.0
num_lab_procedures	101766.0	4.309564e+01	1.967436e+01	1.0	31.0	44.0	5.700000e+01	132.0
num_procedures	101766.0	1.339730e+00	1.705807e+00	0.0	0.0	1.0	2.000000e+00	6.0
num_medications	101766.0	1.602184e+01	8.127566e+00	1.0	10.0	15.0	2.000000e+01	81.0
number_outpatient	101766.0	3.693572e-01	1.267265e+00	0.0	0.0	0.0	0.000000e+00	42.0
number_emergency	101766.0	1.978362e-01	9.304723e-01	0.0	0.0	0.0	0.000000e+00	76.0
number_inpatient	101766.0	6.355659e-01	1.262863e+00	0.0	0.0	0.0	1.000000e+00	21.0
number_diagnoses	101766.0	7.422607e+00	1.933600e+00	1.0	6.0	8.0	9.000000e+00	16.0

In the continues features we noticed that encounter ID and patient NBR have a wide range of values when you look at the standard deviation. So, we will investigate that to see if we can drop those features.

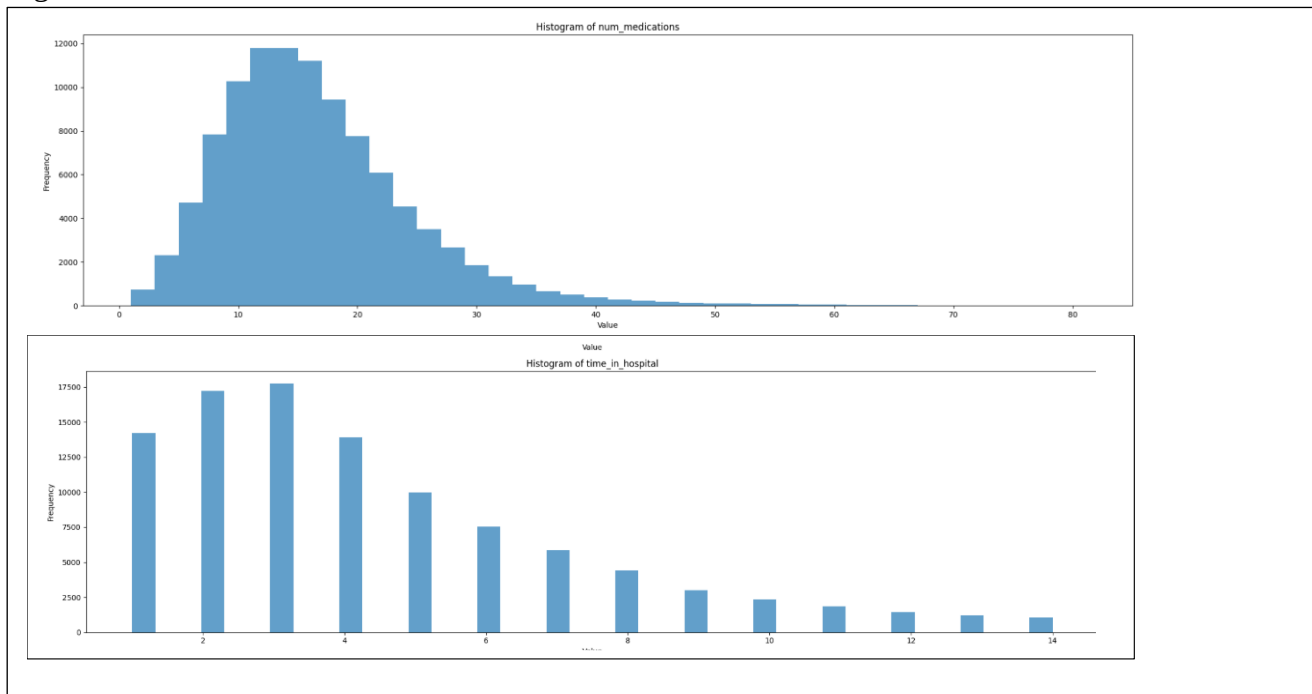
Table 2. Data Quality Report for Categorical Features

Feature	Count	% Miss	Card.	Mode	Mode Freq.	Mode %	2nd Mode	2nd Mode Freq	2nd Mode %
weight	101766	96.858479	10	[75-100]	1336	1.312816	[50-75]	897	0.88
max_glu_serum	101766	94.746772	4	Norm	2597	2.551933	>200	1485	1.46
A1Cresult	101766	83.277322	4	>8	8216	8.073423	Norm	4990	4.90
medical_specialty	101766	49.082208	73	InternalMedicine	14635	14.381031	Emergency/Trauma	7565	7.43
payer_code	101766	39.557416	18	MC	32439	31.876069	HM	6274	6.17
race	101766	2.233555	6	Caucasian	76099	74.778413	AfricanAmerican	19210	18.88
diag_3	101766	1.398306	790	250	11555	11.354480	401	8289	8.15
diag_2	101766	0.351787	749	276	6752	6.634829	428	6662	6.55
diag_1	101766	0.020636	717	428	6862	6.742920	414	6581	6.47
gender	101766	0.002948	3	Female	54708	53.758623	Male	47055	46.24
metformin-rosiglitazone	101766	0.000000	2	No	101764	99.998035	Steady	2	0.00
insulin	101766	0.000000	4	No	47383	46.560737	Steady	30849	30.31
troglitazone	101766	0.000000	2	No	101763	99.997052	Steady	3	0.00
tolazamide	101766	0.000000	3	No	101727	99.961677	Steady	38	0.04
examide	101766	0.000000	1	No	101766	100.000000	na	na	na
citoglipton	101766	0.000000	1	No	101766	100.000000	na	na	na
glipizide-metformin	101766	0.000000	2	No	101753	99.987226	Steady	13	0.01
glyburide-metformin	101766	0.000000	4	No	101060	99.306252	Steady	692	0.68
metformin-pioglitazone	101766	0.000000	2	No	101765	99.999017	Steady	1	0.00
glimepiride-pioglitazone	101766	0.000000	2	No	101765	99.999017	Steady	1	0.00
diabetesMed	101766	0.000000	2	Yes	78363	77.003125	No	23403	23.00
acarbose	101766	0.000000	4	No	101458	99.697345	Steady	295	0.29
change	101766	0.000000	2	No	54755	53.804807	Ch	47011	46.20
miglitol	101766	0.000000	4	No	101728	99.962659	Steady	31	0.03
glipizide	101766	0.000000	4	No	89080	87.534147	Steady	11356	11.16
rosiglitazone	101766	0.000000	4	No	95401	93.745455	Steady	6100	5.99
pioglitazone	101766	0.000000	4	No	94438	92.799167	Steady	6976	6.85
tolbutamide	101766	0.000000	2	No	101743	99.977399	Steady	23	0.02
glyburide	101766	0.000000	4	No	91116	89.534815	Steady	9274	9.11
acetoheamide	101766	0.000000	2	No	101765	99.999017	Steady	1	0.00
glimepiride	101766	0.000000	4	No	96575	94.899082	Steady	4670	4.59
chlorpropamide	101766	0.000000	4	No	101680	99.915492	Steady	79	0.08
nateglinide	101766	0.000000	4	No	101063	99.309200	Steady	668	0.66
repaglinide	101766	0.000000	4	No	100227	98.487707	Steady	1384	1.36
metformin	101766	0.000000	4	No	81778	80.358862	Steady	18346	18.03
admission_source_id	101766	0.000000	17	7	57494	56.496276	1	29565	29.05
discharge_disposition_id	101766	0.000000	26	1	60234	59.188727	3	13954	13.71
admission_type_id	101766	0.000000	8	1	53990	53.053083	3	18869	18.54
age	101766	0.000000	10	[70-80]	26068	25.615628	[60-70]	22483	22.09
readmitted	101766	0.000000	3	NO	54864	53.911916	>30	35545	34.93

For the categorical features we can see that in total there are a total of 10 features with missing data. We can also see that for the race feature the mode is Caucasian with 74% of the data consisting of it and second mode being African American with 18.88% of the data. Another interesting thing that we saw is that the examide and citoglipton have a cardinality of one with their mode taking 100% of the data.

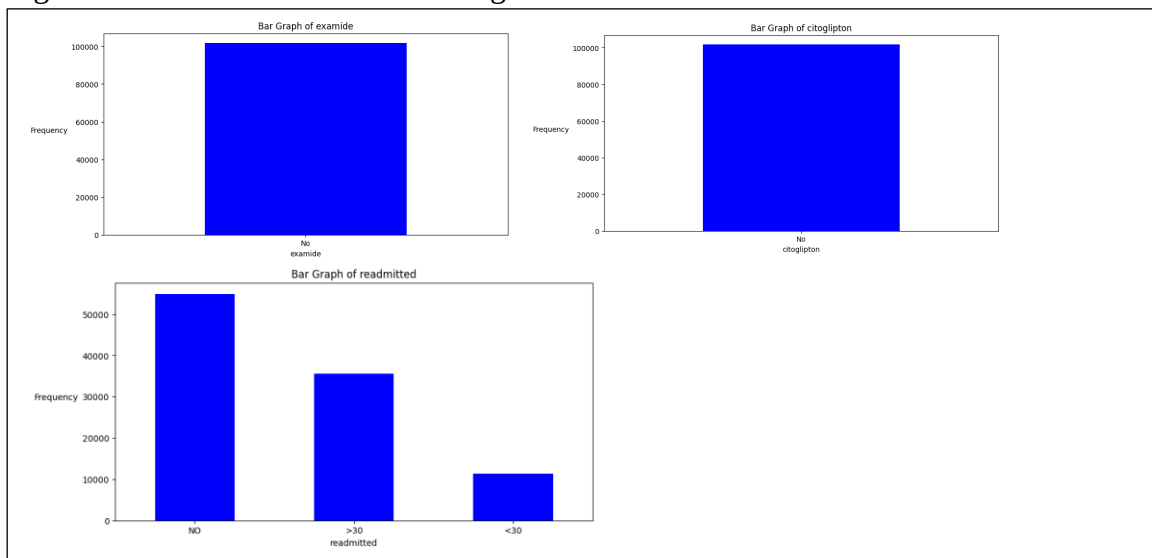
Next, we moved exploring some of the data. In figure 1 we can see that Number of medications and time in hospital are both unimodal (skewed right) types of distribution.

Figure 1. Visualizations of some Continuous Features in Dataset



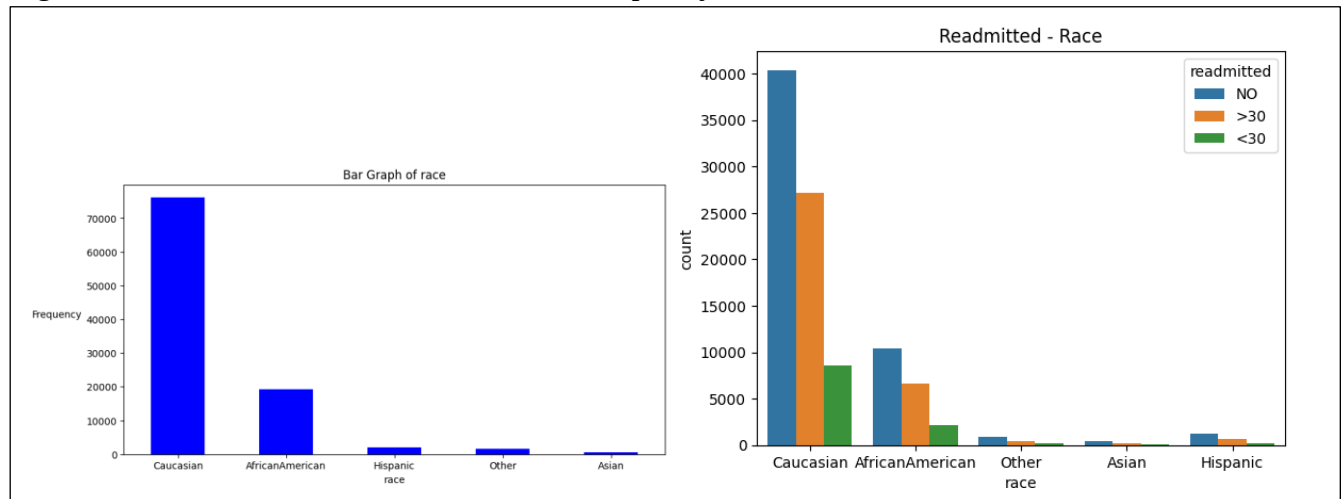
In figure 2 we visualized some of the categorical features and examide and citoglipton caught our eyes once again since they stay the same for every instance of the data. We also added our target feature to the graphs we show so we can show how the target feature is present in the data.

Figure 2. Visualizations of some Categorical Features in Dataset



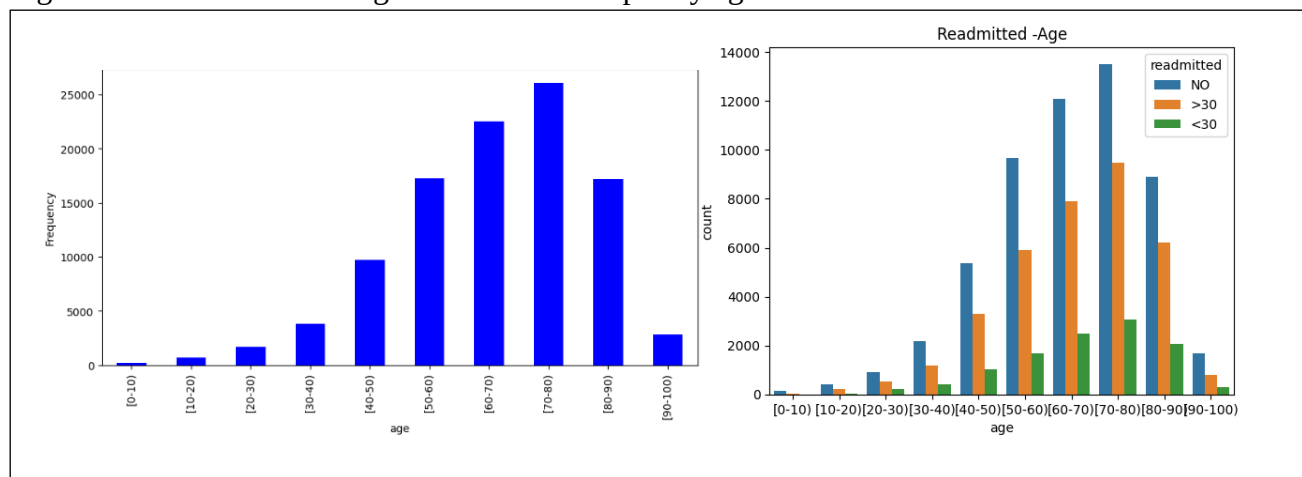
For figure 3 we can see that the distribution of race with the readmittance of each specific race. We can conclude that any readmissions pattern will follow across race since the readmittance rate seems to be proportional across the race categories.

Figure 3. Visualizations of race and readmitted split by race



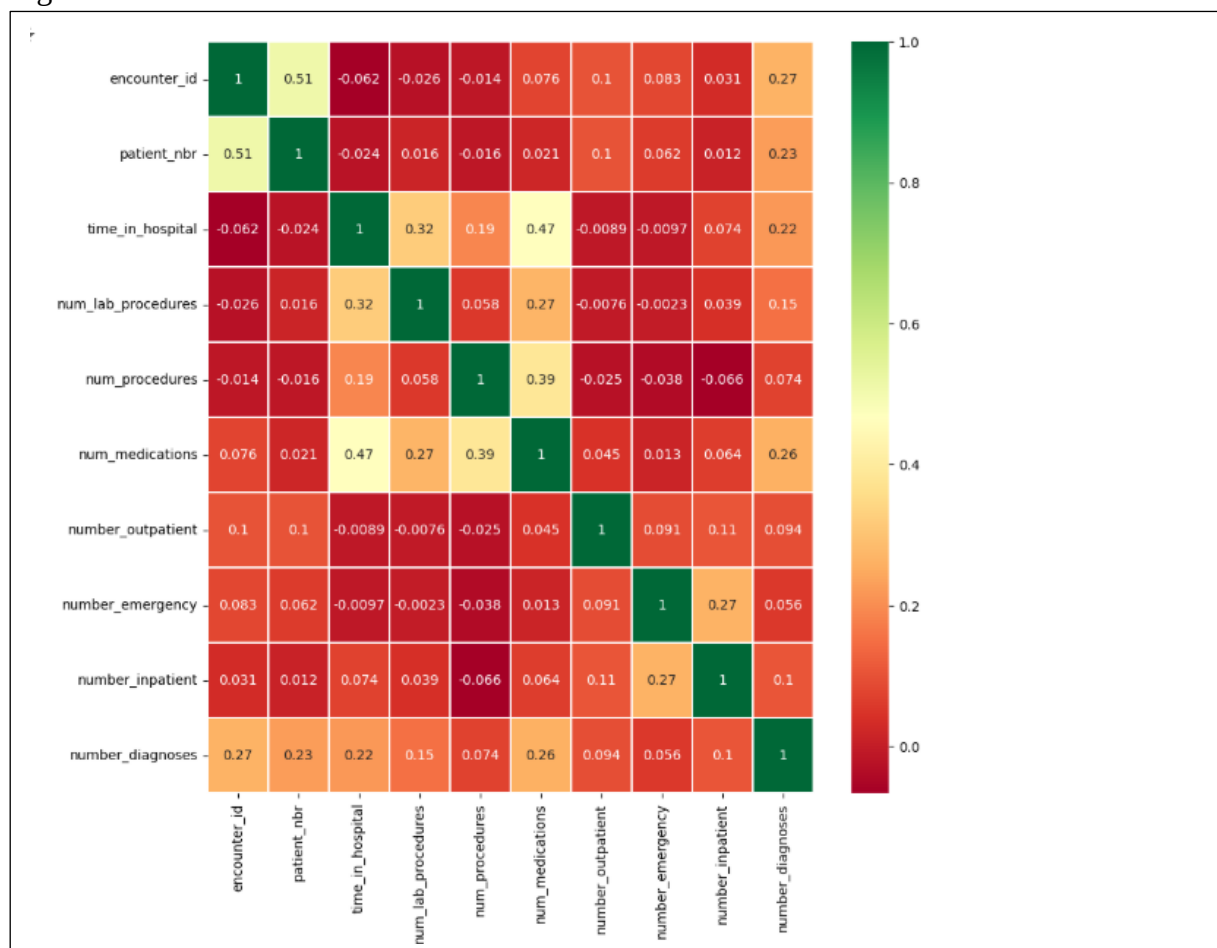
For figure 4 we can see that the distribution of age with the readmittance of each specific age. We can conclude that any readmissions pattern will follow across age groups since the readmittance rate seems to be proportional across the race age category as well.

Figure 4. Visualizations of age and readmitted split by age



In figure 5 we can see the highest correlation between encounter ID and patient NBR which might get dropped from the dataset once we investigate the meaning and purpose of each feature.

Figure 5.



2.2 Missing Values and Outliers

2.2.1 Data Cleaning and Dropping Features

1. Drop irrelevant or constant columns:

```
#drop examide citoglipton bc cardinality is 1 and all instances have the same value
#drop weight max_glu_serum A1Cresult medical_specialty payer_code bc of percentage of missing data is too high
#drop patient_nbr identifies patient and if patient has visited multiple times then they would biased the data. So keep first
#Drop patient encounter_id and patient_nbr bc it is not needed bc identifying individual patients can be do by index number

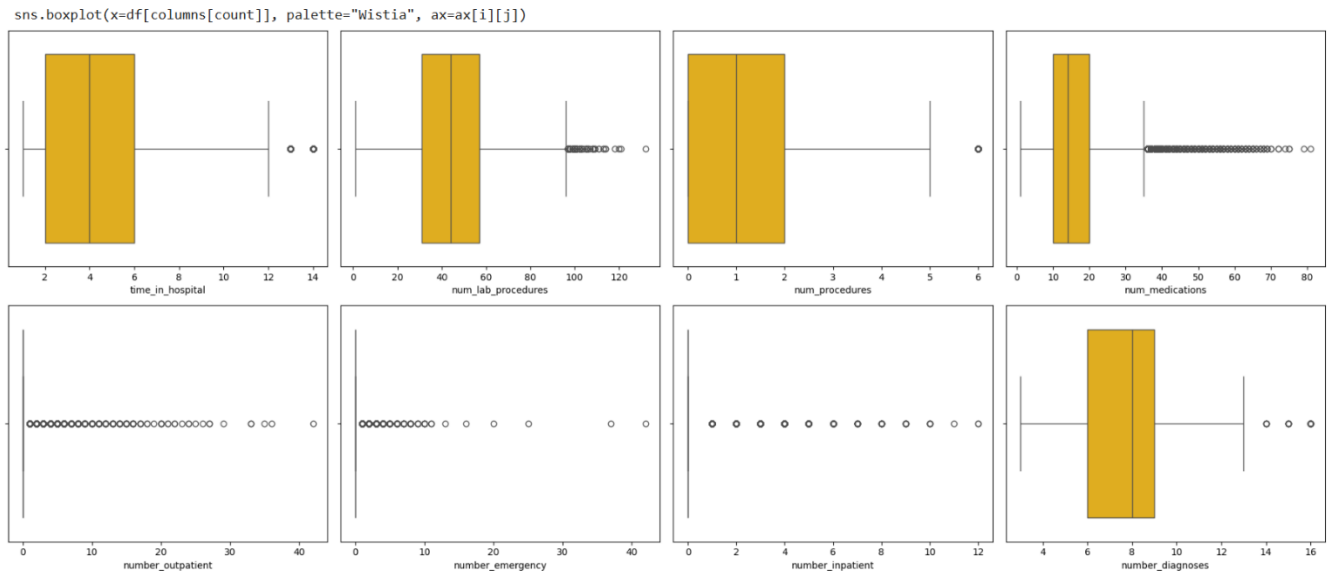
drop_list = ['examide', 'citoglipton', 'weight', 'max_glu_serum', 'A1Cresult', 'medical_specialty', 'payer_code']
df.drop(drop_list, axis=1, inplace=True)
```

Drops columns with either constant values (no analytical value) or a high percentage of missing data, which would introduce bias or reduce model quality.

2.2.2 Outlier Detection

1. Boxplot visualization for numeric columns:

Plots boxplots for numeric features to visually identify outliers for possible handling, ensuring data consistency.



Plots boxplots for numeric features to visually identify outliers for possible handling, ensuring data consistency.

2. Remove gender entries with "unknown" values:

```
# drop where gender is not known|  
df.dropna(subset=['gender'], how='all', inplace = True)
```

Ensures only valid gender data is retained, removing rows with missing gender information for better model integrity.

2.2.3 Handling Missing Data

1. Fill missing values in 'race':

Replaces missing values in the 'race' column with the mode (most common category) to minimize information loss.

```
# fill race with mode  
df["race"].fillna(df["race"].mode()[0], inplace = True)
```

2. Drop rows where any diagnosis (diag_1, diag_2, diag_3) is missing:

```
# drop diagnosis when not known

df.dropna(subset=['diag_1'], how='all', inplace = True)
df.dropna(subset=['diag_2'], how='all', inplace = True)
df.dropna(subset=['diag_3'], how='all', inplace = True)
```

Ensures that rows with complete lack of diagnostic data are removed, as these are critical to analysis.

2.2.4 Duplicate Removal:

1. Drop duplicate patient records:

Retains only the first encounter of each patient to prevent data leakage or bias from multiple encounters. Drops patient identifiers to prevent models from inadvertently "learning" patient-specific patterns, focusing on generalizable features.

```
# Drop duplicates based on 'patient_nbr'

df.drop_duplicates(subset='patient_nbr', keep='first', inplace=True)

# Drop specified columns

columns_to_drop = ['encounter_id', 'patient_nbr']
df.drop(columns_to_drop, axis=1, inplace=True)
```

2.2.5 Handling Outliers

1. Outlier treatment using IQR:

```
for value in con_list:
    qr1 = df[f'{value}'].quantile(0.25)
    qr3 = df[f'{value}'].quantile(0.75)
    iqr = qr3 - qr1
    upper_bound = qr3 + 1.5 * iqr
    lower_bound = qr1 - 1.5 * iqr

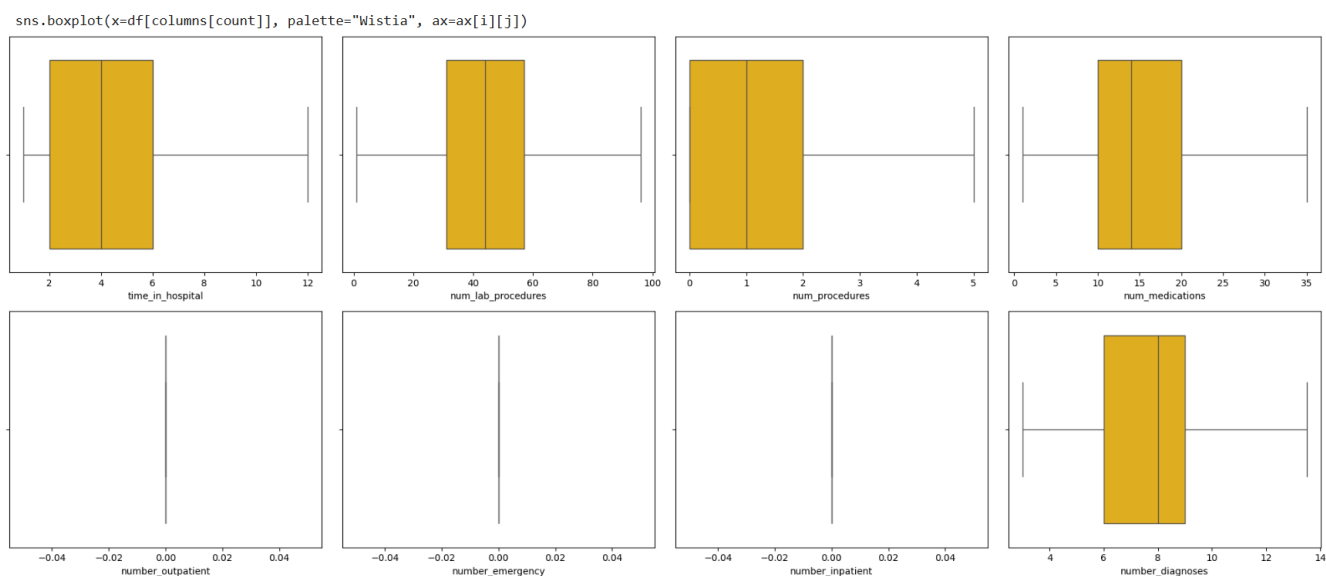
    # upper_bound = qr3 + 2.0 * iqr
    # lower_bound = qr1 - 2.0 * iqr

    # Clamps values within the IQR bounds
    df[f'{value}'] = df[f'{value}'].apply(lambda x: lower_bound if x < lower_bound else upper_bound if x > upper_bound else x)

df_for_bp = df._get_numeric_data()
```

Clamps extreme values in numeric features to IQR bounds, minimizing the impact of outliers while preserving data range consistency.

Visual validation to ensure outlier handling was applied effectively and the data is now within reasonable bounds.



2.3 Normalization

After cleaning and initial transformations, we normalized the numerical features in our dataset to ensure all variables were on a comparable scale, which is crucial for many machine learning algorithms to perform optimally. We implemented a min-max normalization approach to scale all numeric features to a range between 0 and 1.

2.3.1 Normalization Method

We employed min-max normalization using the following formula:

$$x_{normalized} = (x - x_{min}) / (x_{max} - x_{min}) * (new_max - new_min) + new_min$$

```
def min_max_normalization(series, new_min, new_max):
    min_val = series.min()
    max_val = series.max()
    if max_val == min_val:
        return ([float(new_min)] * len(series)) # Assign all values to new_min if all values in series are the same
    return (series - min_val) / (max_val - min_val) * (new_max - new_min) + new_min

# Apply Min-Max Normalization for each continuous feature
for column in df.select_dtypes(include=[np.number]).columns:
    print(column)
    df[f"{column}_norm"] = min_max_normalization(df[column], 0, 1)
```

2.3.2 Normalized Features

The following numeric features were normalized:

- **Demographic:** *gender*
- **Hospital Visit Information:** *time_in_hospital*

- **Medical Procedures and Tests:**

- *num_lab_procedures*
- *num_procedures*
- *num_medications*

- **Patient History:**

- *number_outpatient*
- *number_emergency*
- *number_inpatient*
- *number_diagnoses*

2.3.3 Impact of Normalization

The normalization process ensures that:

- All numeric features now fall within the range [0,1]
- No feature range dominates the others due to different scales
- The relative relationships between values within each feature are preserved
- The model can treat all numeric features equally during training

Binary categorical features (such as *gender*, *diabetesMed*, *medication indicators*, *drugs administered and readmitted*) were left as 0/1 encoding since they were already effectively normalized after doing transformations and mapping the ones we could to 0s and 1s.

2.4 Transformations

In the dataset, we have several Nominal attributes. We transformed these attributes to Numerical values. The following transformations took place for the attributes:

- **Readmitted:** This column is our target feature. It is about "days to inpatient readmission". If the patient was readmitted in less than 30 days "<30". If the patient was readmitted in more than 30 days ">30". If there is no record "NO". We decided to reduce these values to two and map them according to the following rule: 0 for "NO" and ">30", 1 for "<30"

```
# transform readmissions into continuous data
df.readmitted = [1 if each=='<30' else 0 for each in df.readmitted]
```

- **Age:** The attribute was given in range values like [0,10), [10,20) , ..., up to [90,100). These values were mapped to numbers 1 to 10 in a sequential order.

```
def replace_age_ranges(feature):
    age_ranges = ['[0-10)', '[10-20)', '[20-30)', '[30-40)', '[40-50)', '[50-60)', '[60-70)', '[70-80)', '[80-90)', '[90-100)']
    values = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    # The function iterates through the age_ranges and replaces each occurrence in the 'feature' with the corresponding numerical value.
    for i, age_range in enumerate(age_ranges):
        feature = feature.replace(age_range, values[i])

    return feature
```

- **Diag_1, Diag_2, Diag_3:** These attributes represented the primary diagnosis (coded as first three digits of ICD9); 848 distinct values, Secondary diagnosis (coded as first three digits of ICD9); 923 distinct values, Additional secondary diagnosis (coded as first three digits of ICD9); 954 distinct values respectively. We used the research paper ([link here](#)) associated with the dataset to map out these distinct numerical values for each diagnosis into fewer hierarchical group names. The mapping took places referencing this table:

TABLE 2: Values of the primary diagnosis in the final dataset. In the analysis, groups that covered less than 3.5% of encounters were grouped into “other” category.

Group name	icd9 codes	Number of encounters	% of encounter	Description
Circulatory	390–459, 785	21,411	30.6%	Diseases of the circulatory system
Respiratory	460–519, 786	9,490	13.6%	Diseases of the respiratory system
Digestive	520–579, 787	6,485	9.3%	Diseases of the digestive system
Diabetes	250.xx	5,747	8.2%	Diabetes mellitus
Injury	800–999	4,697	6.7%	Injury and poisoning
Musculoskeletal	710–739	4,076	5.8%	Diseases of the musculoskeletal system and connective tissue
Genitourinary	580–629, 788	3,435	4.9%	Diseases of the genitourinary system
Neoplasms	140–239	2,536	3.6%	Neoplasms
	780, 781, 784, 790–799	2,136	3.1%	Other symptoms, signs, and ill-defined conditions
	240–279, without 250	1,851	2.6%	Endocrine, nutritional, and metabolic diseases and immunity disorders, without diabetes
	680–709, 782	1,846	2.6%	Diseases of the skin and subcutaneous tissue
	001–139	1,683	2.4%	Infectious and parasitic diseases
Other (17.3%)	290–319	1,544	2.2%	Mental disorders
	E–V	918	1.3%	External causes of injury and supplemental classification
	280–289	652	0.9%	Diseases of the blood and blood-forming organs
	320–359	634	0.9%	Diseases of the nervous system
	630–679	586	0.8%	Complications of pregnancy, childbirth, and the puerperium
	360–389	216	0.3%	Diseases of the sense organs
	740–759	41	0.1%	Congenital anomalies

The code used to transform the values is as follows:

```
def transformCategory(value):
    if value>=390 and value<=459 or value==785:
        category = 'Circulatory'
    elif value>=460 and value<=519 or value==786:
        category = 'Respiratory'
    elif value>=520 and value<=579 or value==787:
        category = 'Digestive'
    elif value==250:
        category = 'Diabetes'
    elif value>=800 and value<=999:
        category = 'Injury'
    elif value>=710 and value<=739:
        category = 'Musculoskeletal'
    elif value>=580 and value<=629 or value==788:
        category = 'Genitourinary'
    elif value>=140 and value<=239 :
        category = 'Neoplasms'
    elif value==1:
        category = 'NAN'
    else :
        category = 'Other'

    return category
```

- **23 features for Medications:** The dataset contains 23 distinct medicine names as its attributes. 2 of these attributes: examide and citoglipton have only “No” values for all instances of the data. Hence these features were dropped. The other columns had values: “up” if the dosage was

increased during the encounter, “down” if the dosage was decreased, “steady” if the dosage did not change, and “no” if the drug was not prescribed. These values were mapped using the rule: 'No' to 0 and other values to 1

```
medicines = ['metformin', 'repaglinide', 'nateglinide', 'chlorpropamide', 'glimepiride', 'acetohexamide', 'glipizide', 'glyburide',
             'tolbutamide', 'pioglitazone', 'rosiglitazone', 'acarbose', 'miglitol', 'troglitazone', 'tolazamide', 'insulin',
             'glyburide-metformin', 'glipizide-metformin', 'glimepiride-pioglitazone', 'metformin-rosiglitazone', 'metformin-pioglitazone']

for col in medicines:
    print(col)
    df[col] = df[col].apply(lambda x: 0 if x == 'No' else 1)
```

- Gender: Gender was mapped to numerical value using the rule: 0 for ‘Female’ and 1 ‘Male’.

```
df['gender'] = df['gender'].apply(lambda x: 0 if x == 'Female' else 1)
```

- Change: Change attribute was mapped using the rule: 0 for ‘No’ and 1 for other.

```
df['change'] = df['change'].apply(lambda x: 0 if x == 'No' else 1)
```

- DiabetesMed: DiabetesMed attribute was mapped using the rule: 0 for ‘No’ and 1 for other.

```
df['diabetesMed'] = df['diabetesMed'].apply(lambda x: 0 if x == 'No' else 1)
```

- Race: Race attribute was labelled to numerical value using LabelEncoder from sklearn library. Accordingly, we have 6 distinct values for race assigned 0 to 5.

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
df['race'] = encoder.fit_transform(df['race'])
```

After age range, diagnosis_1, diagnosis_2, diagnosis_3 and the race features got transformed to numerical data, we decided to turn it back to categorical data while keeping the transformation for readability since using the new values as continuous did not make sense since they are nominal data.

```
# change race back to object since it is nominal
df['race'] = df['race'].apply(lambda x: str(x))
```

```
df['age'] = df['age'].apply(lambda x: str(x))
```

```
for diag in diag_list:
    df[diag] = df[diag].apply(transformCategory)
```